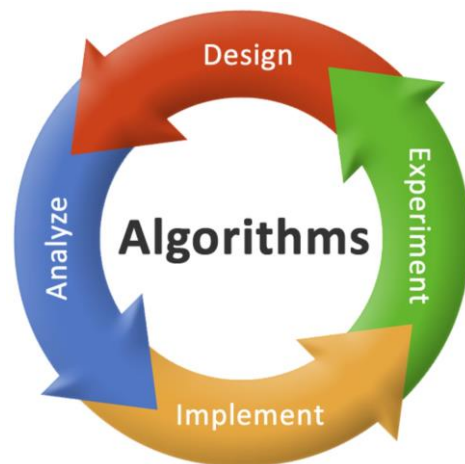


华中科技大学
HUAZHONG UNIVERSITY OF SCIENCE AND TECHNOLOGY



Algorithm Design and Analysis

算法设计与分析

Chapter 1: Asymptotic Analysis | 张乾坤

课程提要

- 算法分析基础
- 渐近分析（ Asymptotic Analysis ）——函数的增长
- 最好、最坏、平均情形复杂度分析

课程提要

- 算法分析基础
- 渐近分析（ Asymptotic Analysis ）——函数的增长
- 最好、最坏、平均情形复杂度分析

算法分析基础

- 分析算法的目的是研究计算某特定问题需要的资源（时间&空间）
- 基本假设：随机访问机（Random-Access Machine, RAM）
 - 单处理器计算模型
 - 指令集：
 - 基本算术指令（加减乘除、取余、取整、指数等）
 - 数据移动指令（装入，存储，复制等）
 - 控制指令（条件转移、子程序调用等）

常数时间可计算

暴力法 (Brute Force)

- 确认每一种可能的输入结果
- 对于长度为 n 的输入，往往需要 2^n 步
- 最自然但往往笨拙



暴力法 (Brute Force)

- 例如：背包问题 (Knapsack Problem)
- n 个物品，每个物品价值为 v_i ，重量为 w_i ，在满足重量不超过 W 的情况下，求价值最大的物品组合



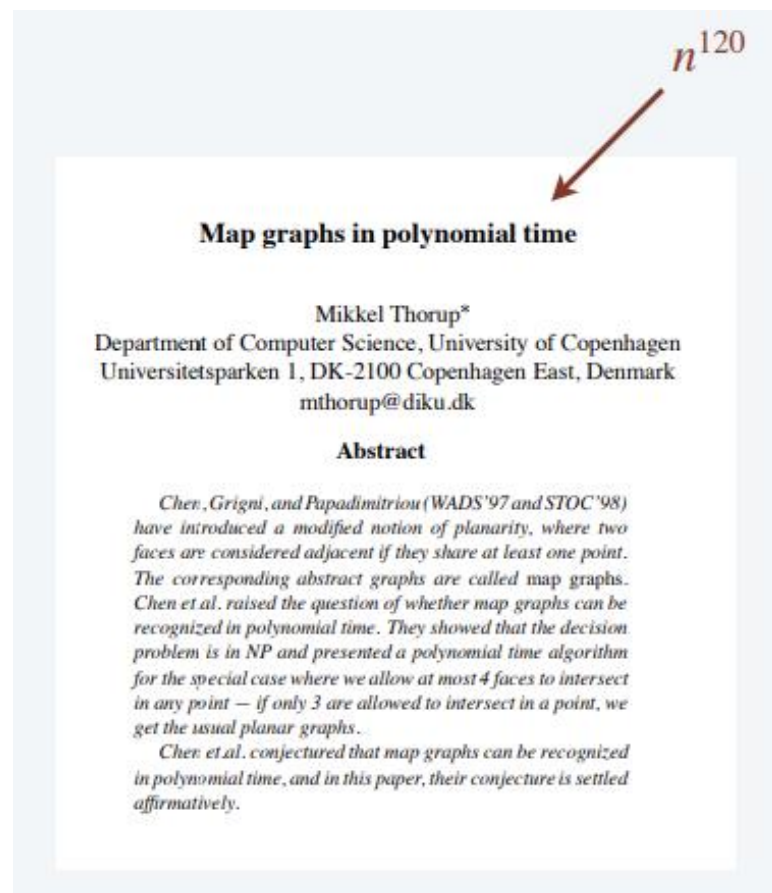
Polynomial Running Time

多项式时间算法——Efficient Algorithm

- 当输入长度翻倍时，算法运行时间最多会慢常数 $c > 0$ 倍
- 存在常数 $a > 0$ 和 $b > 0$ ，满足对任意长度为 n 的输入，算法执行至多 $a \cdot n^b$ 步

Polynomial Running Time

- 真实应用场景
 - ✓ 多项式算法中常数 a 和 b 通常不会太大
 - ✓ 突破暴力法的指数级复杂度通常依赖计算问题的特殊结构
- 也有特例



Polynomial Running Time

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

课程提要

- 算法分析基础
- 渐近分析（ Asymptotic Analysis ）——函数的增长
- 最好、最坏、平均情形复杂度分析

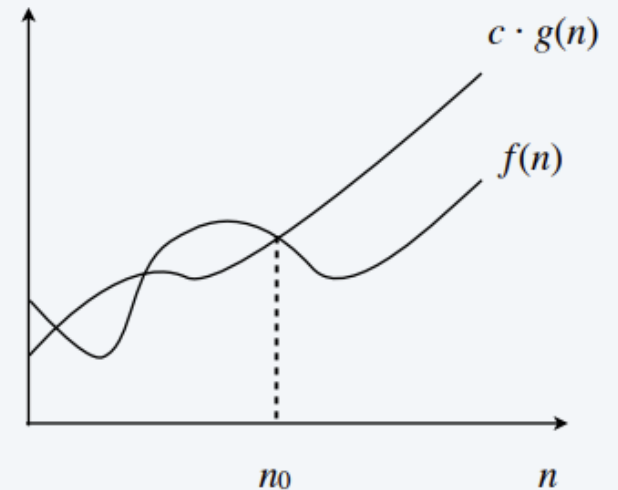
大O表示法 (Big O Notation)

- 渐进上界 (Asymptotic upper bound) “最差情形”

Upper bounds. $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Ex. $f(n) = 32n^2 + 17n + 1$.

- $f(n)$ is $O(n^2)$. $\longleftarrow$ choose $c = 50, n_0 = 1$
- $f(n)$ is neither $O(n)$ nor $O(n \log n)$.



大O表示法 (Big O Notation)

• 思考题：设 $f(n) = 314n^2 + 1592n \log_2 n + 6535$ ，下列说法正确的是？

A. $f(n)$ 是 $O(n^2)$

B. $f(n)$ 是 $O(n^3)$

C. A和B都对

D. A和B都不对

大O表示法 (Big O Notation)

- 一个不规范的符号混用: $O(g(n))$ 是一个集合, 但通常使用 $f(n) = O(g(n))$, 而不用 $f(n) \in O(g(n))$
- 例: $g_1(n) = 3n^3$, $g_2(n) = 2n^2 \rightarrow g_1(n) = O(n^3)$ 且 $g_2(n) = O(n^3)$
- 但 $g_1(n) \neq g_2(n)$

大O表示法的性质

- 自反性 Reflexivity. f is $O(f)$.

- 常数 Constants. If f is $O(g)$ and $c > 0$, then cf is $O(g)$.

- 乘积 Products. If f_1 is $O(g_1)$ and f_2 is $O(g_2)$, then $f_1 f_2$ is $O(g_1 g_2)$.

Pf.

- $\exists c_1 > 0$ and $n_1 \geq 0$ such that $0 \leq f_1(n) \leq c_1 \cdot g_1(n)$ for all $n \geq n_1$.
- $\exists c_2 > 0$ and $n_2 \geq 0$ such that $0 \leq f_2(n) \leq c_2 \cdot g_2(n)$ for all $n \geq n_2$.
- Then, $0 \leq f_1(n) \cdot f_2(n) \leq \underbrace{c_1 \cdot c_2}_c \cdot g_1(n) \cdot g_2(n)$ for all $n \geq \underbrace{\max \{n_1, n_2\}}_{n_0}$. ■

- 求和 Sums. If f_1 is $O(g_1)$ and f_2 is $O(g_2)$, then $f_1 + f_2$ is $O(\max \{g_1, g_2\})$.

ignore lower-order terms

- 传递性 Transitivity. If f is $O(g)$ and g is $O(h)$, then f is $O(h)$.

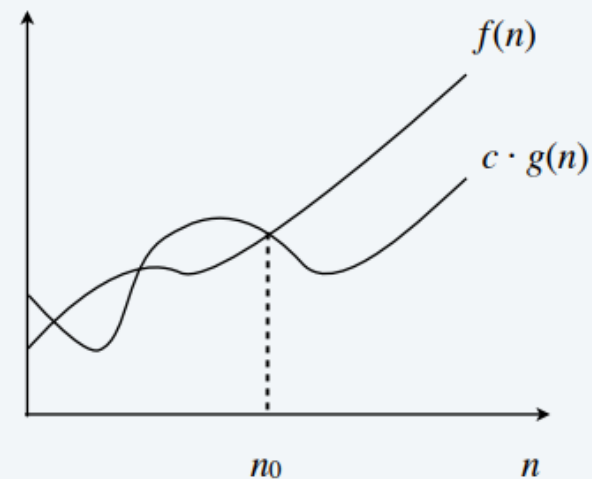
大Omega表示法 (Big Omega Notation)

- 渐进下界 (Asymptotic **lower** bound)

Lower bounds. $f(n)$ is $\Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \geq c \cdot g(n) \geq 0$ for all $n \geq n_0$.

Ex. $f(n) = 32n^2 + 17n + 1$.

- $f(n)$ is both $\Omega(n^2)$ and $\Omega(n)$. ← choose $c = 32, n_0 = 1$
- $f(n)$ is not $\Omega(n^3)$.



大Omega表示法 (Big Omega Notation)

- 渐进下界 (Asymptotic lower bound)



- 任何基于比较的排序算法在最差情况下都需要比较 $\Omega(n \log n)$ 次
- 任何基于比较的排序算法在最差情况下都需要比较至少 $\Omega(n \log n)$ 次

大Omega表示法 (Big Omega Notation)

• 思考题：下列说法和大Omega表示法定义等价的是？

A. $f(n) = \Omega(g(n))$ 当且仅当 $g(n) = O(f(n))$

B. $f(n) = \Omega(g(n))$ 当且仅当存在一个大于0的常数 c ，使得对无穷多 n 满足
 $f(n) \geq c \cdot g(n) \geq 0$

C. A和B都对

D. A和B都不对

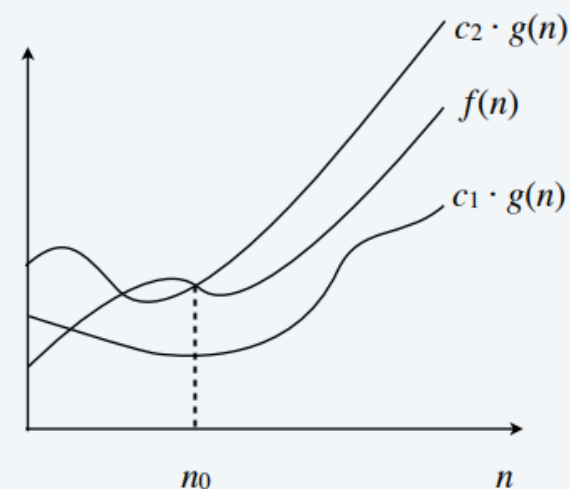
大Theta表示法 (Big Theta Notation)

- 渐进紧界 (Asymptotic **tight** bound)

Tight bounds. $f(n)$ is $\Theta(g(n))$ if there exist constants $c_1 > 0$, $c_2 > 0$, and $n_0 \geq 0$ such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

Ex. $f(n) = 32n^2 + 17n + 1$.

- $f(n)$ is $\Theta(n^2)$. $\leftarrow$ choose $c_1 = 32, c_2 = 50, n_0 = 1$
- $f(n)$ is neither $\Theta(n)$ nor $\Theta(n^3)$.



大Theta表示法 (Big Theta Notation)

• 思考题1: 下列说法和大Theta表示法定义等价的是?

A. $f(n) = \Theta(g(n))$ 当且仅当 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$

B. 若存在一个常数 $0 < c < \infty$, 使得 $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$, 则 $f(n) = \Theta(g(n))$

C. A和B都对

D. A和B都不对

一些重要函数的渐近界

- 多项式: $f(n) = a_0 + a_1n + \cdots + a_mn^m$, 其中 $a_m > 0$, 则 $f(n) = \Theta(n^m)$
- 对数: 对任意 $a > 1$ 和 $b > 1$, $\log_a n = \Theta(\log_b n)$
- 对数 vs. 多项式: 对任意 $a > 1$ 和 $m > 0$, $\log_a n = O(n^m)$
- 指数 vs. 多项式: 对任意 $a > 1$ 和 $m > 0$, $n^m = O(a^n)$

多变量大O表示法

Upper bounds. $f(m, n)$ is $O(g(m, n))$ if there exist constants $c > 0$, $m_0 \geq 0$, and $n_0 \geq 0$ such that $0 \leq f(m, n) \leq c \cdot g(m, n)$ for all $n \geq n_0$ or $m \geq m_0$.

Ex. $f(m, n) = 32mn^2 + 17mn + 32n^3$.

- $f(m, n)$ is both $O(mn^2 + n^3)$ and $O(mn^3)$.
- $f(m, n)$ is $O(n^3)$ if a precondition to the problem implies $m \leq n$.
- $f(m, n)$ is neither $O(n^3)$ nor $O(mn^2)$.

常见的算法运行时间上界

- 运行时间时间：多项式时间 & 指数时间

- 常见的多项式上界函数

$$\checkmark O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3)$$

- 常见的指数上界函数

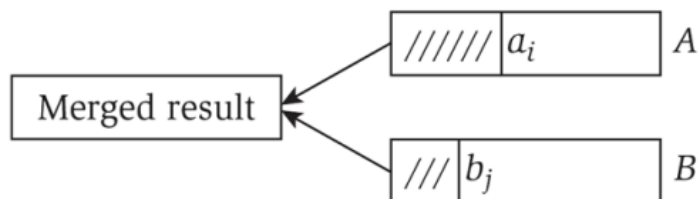
$$\checkmark O(2^n) < O(n!) < O(n^n)$$

常见的算法运行时间上界

- 常数时间: $O(1)$
 - ✓ 条件跳转
 - ✓ 代数运算
 - ✓ 声明或初始化变量
 - ✓ 访问数组的某个元素
 - ✓ 交换数组中的两个元素

常见的算法运行时间上界

- 线性Linear时间: $O(n)$
- 例: 归并排序中合并两个已经排好序的数组



$i \leftarrow 1; j \leftarrow 1.$

WHILE (both lists are nonempty)

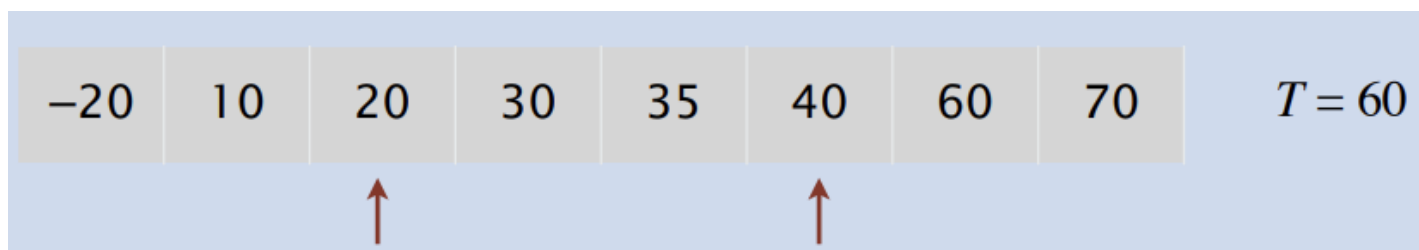
IF ($a_i \leq b_j$) append a_i to output list and increment i .

ELSE append b_j to output list and increment j .

Append remaining elements from nonempty list to output list.

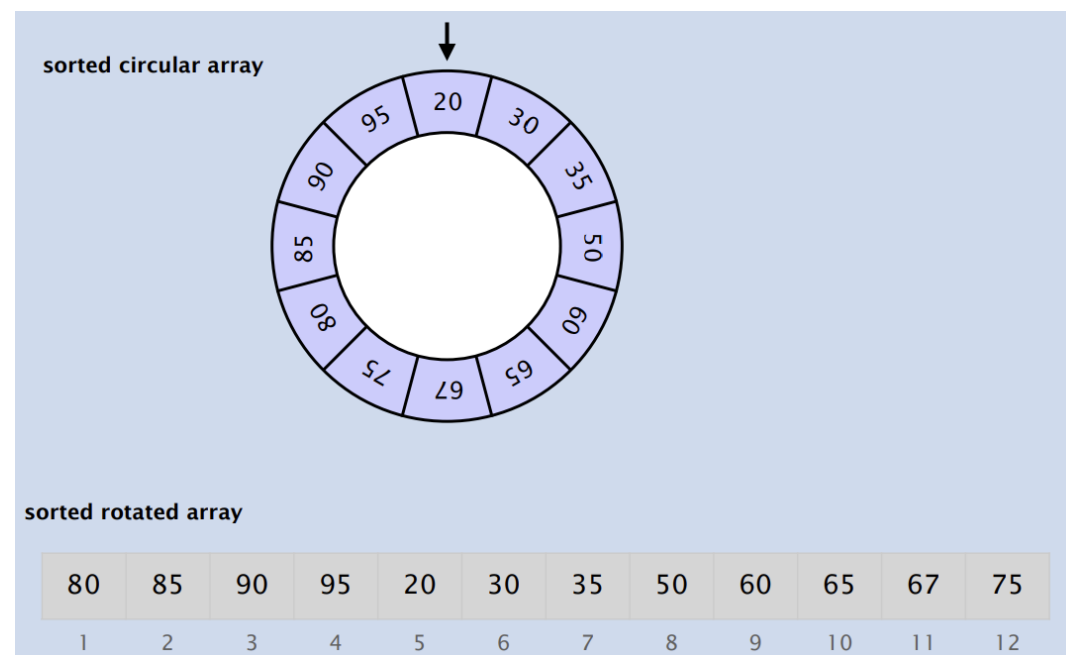
常见的算法运行时间上界

- 线性Linear时间: $O(n)$
- 思考题: 输入一个长度为 n 且已按升序排列的数组和一个整数 T , 求数组中和 T 为的两个元素。



常见的算法运行时间上界

- 对数Logarithmic时间: $O(\log n)$
 - ✓ 例如: 二分查找
 - ✓ 思考题: 循环数组中查找某一元素 x



常见的算法运行时间上界

- Linearithmic时间: $O(n \log n)$

- ✓例如: 排序算法

- ✓思考题: 给定一个长度为 n 的整数数组, 求 $|a - b|$ 的最大值, 其中 a 和 b 是数组中的某两元素。

常见的算法运行时间上界

- 二次Quadratic时间: $O(n^2)$

- ✓例如: 排序算法

- ✓思考题: 给定平面上 n 点, 距离最近的两个点。

- ✓枚举所有的点对

$min \leftarrow \infty.$

FOR $i = 1$ TO n

FOR $j = i + 1$ TO n

$d \leftarrow (x_i - x_j)^2 + (y_i - y_j)^2.$

IF ($d < min$)

$min \leftarrow d.$

常见的算法运行时间上界

- 三次Cubic时间: $O(n^3)$

✓ 3-Sum问题: 输入一个长度为 n 的数组, 求数组中和为0的三个元素。

```
FOR  $i = 1$  TO  $n$ 
  FOR  $j = i + 1$  TO  $n$ 
    FOR  $k = j + 1$  TO  $n$ 
      IF  $(a_i + a_j + a_k = 0)$ 
        RETURN  $(a_i, a_j, a_k)$ .
```

✓ But can we do better? $O(n^2)$ 算法?

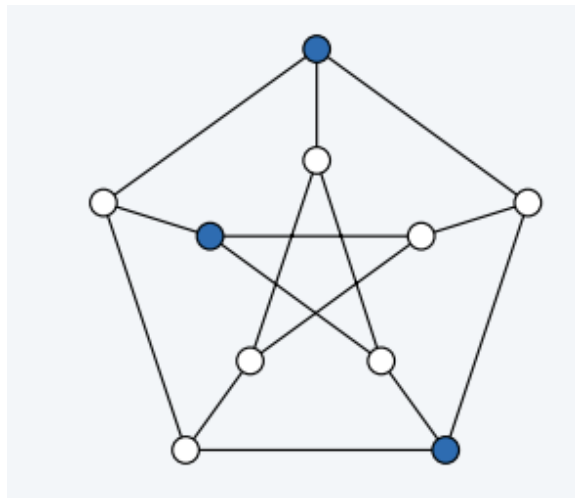
常见的算法运行时间上界

- 多项式Polynomial时间: $O(n^k)$

✓ k -独立集 (Independent Set of Size k) 问题: 输入一张图, 找出 k 个点, 使任意两个点之间没有边。

✓ $O(n^k)$ 算法:

1. 枚举所有 k 个点的子集
2. 判断是否为独立集



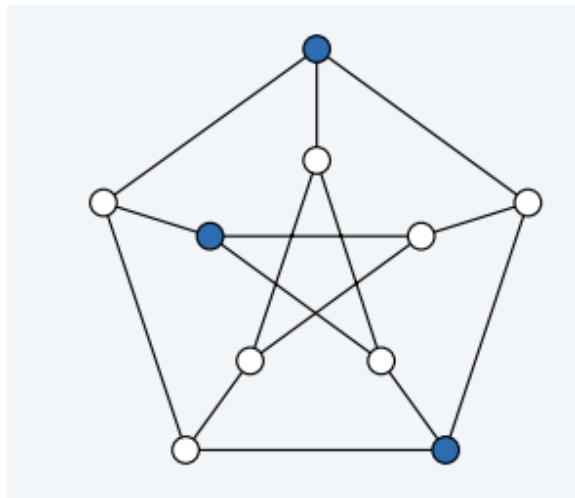
常见的算法运行时间上界

- 指数Exponential时间: $O(2^{n^k})$

✓ (最大) 独立集 (Independent Set) 问题: 输入一张图, 找出满足任意两个点之间没有边的最大的顶点子集。

✓ $O(2^n)$ 算法:

1. 枚举所有顶点的子集
2. 判断是否为独立集



课程提要

- 算法分析基础
- 渐近分析（ Asymptotic Analysis ）——函数的增长
- 最好、最坏、平均情形复杂度分析

最好情形 vs. 最坏情形 vs. 平均情形

- 给定数组中搜索某个元素的下标
- $O(n)$: 遍历整个数组

```
1  //n表示数组array的长度
2  int find(int[] array, int n, int x) {
3      int i=0;
4      int pos=-1;
5      for (; i < n; ++i) {
6          if (array[i] == x) pos = i;
7      }
8      return pos;
9  }
```

最好情形 vs. 最坏情形 vs. 平均情形

- 给定数组中搜索某个元素的下标

```
1 //n表示数组array的长度
2 int find(int[] array,int n,int x){
3     int i = 0;
4     int pos = -1;
5     for (; i < n; ++i){
6         if (array[i] == x){
7             pos = i;
8             break;
9         }
10    }
11    return pos;
12 }
```

最好情形 vs. 最坏情形 vs. 平均情形

- 给定数组中搜索某个元素的下标

```
1 // n表示数组array的长度
2 int find(int[] array, int n, int x){
3     int i = 0;
4     int pos = -1;
5     for (; i < n; ++i){
6         if (array[i] == x){
7             pos = i;
8             break;
9         }
10    }
11    return pos;
12 }
```

- 平均时间复杂度

- 假设变量x在数组中以 $\frac{1}{2^n}$ 概率不出现，若出现则n个位置等概率

$$\begin{aligned} & \bullet 1 \cdot \frac{1}{2^n} + 2 \cdot \frac{1}{2^n} + \dots + n \cdot \frac{1}{2^n} \\ & + n \cdot \frac{1}{2} = \frac{3n+1}{4} = O(n) \end{aligned}$$